CHAPTER 36

Numerical Missing Data - SimpleImputer

Mean Imputation, Median Imputation, Strategy, Statistics

CampusX — 100 Days of Machine Learning
Feature Engineering Series, Videos 31–40

36.1 Introduction to Imputation

When Complete Case Analysis is not appropriate — because too much data would be lost, or because the missingness mechanism is MAR — we turn to **imputation**: replacing missing values with estimated ones. The simplest and most widely used imputation method is **mean imputation** (replace NaN with the column mean) or **median imputation** (replace NaN with the column median).

Scikit-learn provides the `SimpleImputer` class for this purpose. Despite its name, SimpleImputer is a foundational tool that every ML practitioner must understand deeply.

36.2 Mean vs Median Imputation

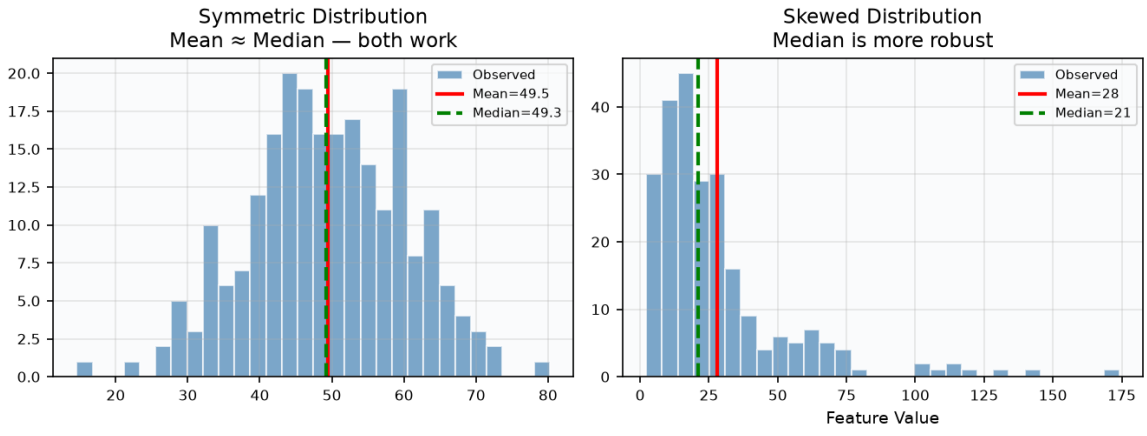


Figure 36.1: Left: for symmetric distributions, mean and median are nearly identical — either works. Right: for skewed distributions, the mean is pulled toward the long tail while the median remains robust. Median imputation is preferred for skewed data.

36.2.1 Mathematical Basis

Mean imputation replaces each missing value x■ with the sample mean of observed values in that feature:

$$\hat{x}_j = \bar{x}_{\text{obs}} = \frac{1}{n_{\text{obs}}} \sum_{i \in \text{obs}} x_{ij} \tag{36.1}$$

Median imputation replaces each missing value with the median of observed values:

$$\hat{x}_j = \text{median}\left(\{x_{ij} : i \in \text{obs}\}\right) \tag{36.2}$$

The mean is sensitive to outliers because it uses all values equally in its computation. The median — the middle value when data is sorted — is robust to outliers because it depends only on the rank order, not the magnitudes.

36.3 Scikit-learn SimpleImputer

Parameter	Default	Description
missing_values	np.nan	The placeholder for missing values
strategy	"mean"	Imputation strategy: "mean", "median", "most_frequent", "constant"
fill_value	None	Value to use when strategy="constant"
copy	True	If True, creates a copy; if False, imputes in place
add_indicator	False	If True, adds binary missingness indicator columns

Table 36.1: SimpleImputer parameters.

Attribute	Description
statistics_	Imputation value for each feature (array of shape [n_features])
n_features_in_	Number of features seen during fit
indicator_	MissingIndicator instance (if add_indicator=True)

Table 36.2: SimpleImputer attributes after fitting.

36.3.1 Full Implementation

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer

# Create dataset with missing numerical values
np.random.seed(42)
n = 100
df = pd.DataFrame({
'age': np.random.normal(35, 10, n),
'income': np.random.lognormal(10.5, 0.8, n), # Right-skewed
'score': np.random.normal(70, 15, n)
})

# Introduce MCAR missing values (15%)
for col in df.columns:
mask = np.random.random(n) < 0.15
df.loc[mask, col] = np.nan

print("=== MISSING VALUES BEFORE ===")
print(df.isnull().sum())
print()

# ■■ Mean Imputation ■■
imputer_mean = SimpleImputer(strategy='mean')
df_mean = pd.DataFrame(
imputer_mean.fit_transform(df),
columns=df.columns
)
print("Mean imputation statistics:", imputer_mean.statistics_)

# ■■ Median Imputation ■■
```

```
imputer_median = SimpleImputer(strategy='median')
df_median = pd.DataFrame(
imputer_median.fit_transform(df),
columns=df.columns
)
print("Median imputation statistics:", imputer_median.statistics_)

print("\n=== MISSING VALUES AFTER ===")
print(f"Mean imputer: {df_mean.isnull().sum().sum()} remaining")
print(f"Median imputer: {df_median.isnull().sum().sum()} remaining")

# Compare distributions
print("\n=== DISTRIBUTION COMPARISON ===")
for col in df.columns:
print(f"\n{col}:")
print(f" Original mean: {df[col].mean():.2f}")
print(f" Mean-imputed: {df_mean[col].mean():.2f}")
print(f" Median-imputed: {df_median[col].mean():.2f}")
print(f" Original median: {df[col].median():.2f}")
print(f" Original skew: {df[col].skew():.2f}")
```

Listing 36.1: Mean vs Median imputation with SimpleImputer.

36.4 Column-Wise Imputation

SimpleImputer computes a **separate** statistic for each feature (column). This is the correct behaviour: each feature has its own distribution, so using a single global mean for all features would be wrong. The `statistics_` attribute reveals the per-feature imputation value after fitting.

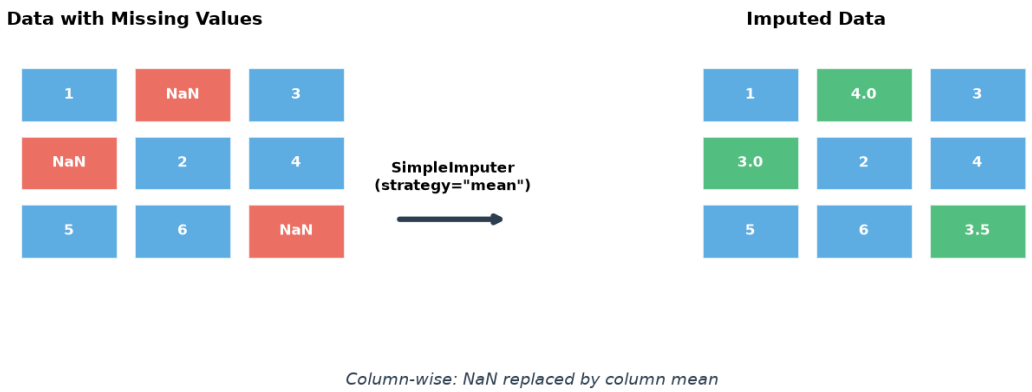


Figure 36.2: SimpleImputer with strategy="mean" replaces each NaN with the mean of its own column. Column 1 mean = 3.0, column 2 mean = 4.0, column 3 mean = 3.5.

36.5 Statistical Implications

Mean Imputation Distorts Variance
Replacing missing values with the column mean **reduces the variance** of the feature. All imputed values are identical (the mean), so they add no spread. This artificial narrowing of the distribution can make relationships appear stronger than they really are and can underestimate standard errors in statistical models.

Median imputation has a similar problem: it reduces variance and can create artificial spikes at the median value. Both methods treat imputed values as if they were observed with certainty, ignoring the uncertainty of the estimate.

Better Alternatives
Methods that preserve the distribution better include: random sample imputation (Chapter 38), KNN imputation (Chapter 39), and iterative/MICE imputation (Chapter 40). These methods produce more realistic imputed values by considering the relationships between features.

36.6 Key Takeaways

★ SimpleImputer replaces missing values with a per-column statistic (mean, median, most_frequent, or constant). Mean works for symmetric data; median is robust to skewness and outliers. Both reduce variance — a limitation to remember when interpreting model results.

36.7 Interview Questions

1. Why is median preferred over mean for skewed distributions?
2. What does SimpleImputer.statistics_ contain after fitting?
3. How does mean imputation affect the variance of a feature?
4. Can SimpleImputer handle categorical data? Which strategy would you use?
5. What is the difference between fit() and transform() in SimpleImputer?